

Question 1

Correct

Mark 2.00 out of 2.00

For each of the following techniques describe if it has been introduced to reduce neural networks overfitting or to enhance the performance of standard backpropagation. You can interpret the question as "The technique <name_of_technique> deals with <the Overfitting issue / Backpropagation issues>."

Overfitting	Backpropagation		
☒	☐	Weight decay	✓ Being a regularization techniques it reduces the number of effective parameters of the net
☒	☐	Dropout	✓ It perform stochastic regularization
☐	☒	Xavier initialization	✓ It reduces the risk of learning instabilities
☐	☒	Leaky ReLU (as an improvement of ReLU)	✓ They allow the rescue of dying neurons stuck due to the zero gradient issue
☐	☒	Shortcut connections	✓ The reduce the issue of vanishing gradient which might happen with deep networks
☒	☐	Data augmentation	✓
☒	☐	Early stopping	✓ Was introduced to prevent overfitting by monitoring it
☐	☒	Batch normalization	✓ It normalizes the parameters of the networks improving the stability of gradient descent

Weight decay: Overfitting

Dropout: Overfitting

Xavier initialization: Backpropagation

Leaky ReLU (as an improvement of ReLU): Backpropagation

Shortcut connections: Backpropagation

Data augmentation: Overfitting

Early stopping: Overfitting

Batch normalization: Backpropagation

Question 2

Partially correct

Mark 1.50 out of 2.00

We are in the general setting of classification, which of the following statements holds true (independently from the others).

- ☒ a. I use as many outputs as the number of classes and the softmax activation function ✔ Yes, classical multiclass scenario
- ☐ b. I use the mean absolute precision loss function
- ☒ c. I use the categorical cross-entropy loss function ✔ Yes it is the standard loss for multi class classification problems
- ☐ d. I use as many outputs nodes as the number of classes and each has a logistic (sigmoid) activation function
- ☒ e. I regularize the model by adding the sum of squares of the weights (L2 regularization) to the loss function ✔ Yes this is typical of weight decay
- ☐ f. I use as many outputs as the classes and linear activation functions

Your answer is partially correct.

You have correctly selected 3.

The correct answers are:

I use the categorical cross-entropy loss function,

I use as many outputs as the number of classes and the softmax activation function,

I use as many outputs nodes as the number of classes and each has a logistic (sigmoid) activation function,

I regularize the model by adding the sum of squares of the weights (L2 regularization) to the loss function

Question 3

Partially correct

Mark 0.75 out of 1.00

Neural networks are trained, most of the time, via some variation of the gradient descent algorithm, also known as backpropagation, designed in 1986 by David E. Rumelhart.

Among the following statements mark which one is true and which one is false ... you cannot leave the question unanswered.

True	False		
☐	☒	A key parameter of gradient descend is the learning rate, if too low it might make the algorithm diverge	☒ No, it is the other way round
☐	☒	In recurrent neural networks, you use backpropagation through time, however, this conceptually (i.e., independently from the specific implementation) requires, all sequences in the training set to have the same length, even with vanilla SGD, i.e., with a batch size equal to 1.	☒ No, you can unroll each sequence for its own length and then backpropagate. The use of padding or truncation at the batch level is for parallelism.
☒	☐	In feedforward neural networks you can minimize the effect of vanishing gradient via skip connections	☐ Yes, they let the gradient flow untouched skipping the layers
☒	☐	To improve algorithm resilience to noisy loss functions, i.e., loss functions with lots of local minima, you can add a momentum term	☒ Yes it is the reason it was originally introduced for

A key parameter of gradient descend is the learning rate, if too low it might make the algorithm diverge: False

In recurrent neural networks, you use backpropagation through time, however, this conceptually (i.e., independently from the specific implementation) requires, all sequences in the training set to have the same length, even with vanilla SGD, i.e., with a batch size equal to 1.: False

In feedforward neural networks you can minimize the effect of vanishing gradient via skip connections: True

To improve algorithm resilience to noisy loss functions, i.e., loss functions with lots of local minima, you can add a momentum term : True

Question 4

Complete

Mark 4.00 out of 4.00

The Transformer architecture has been introduced to overcome the parallelism bottleneck of recurrent architectures. Regarding the Transformer architecture, answer the following questions with motivations.

- a) What is the idea behind self-attention?
- b) What are Q, K, and V in the self-attention mechanism?
- c) What is masked self-attention?
- d) What is positional encoding?

a) The goal of self-attention mechanism in transformer is to allow encoder and decoder to focus on the important and relevant parts of the context when processing the current token, by assigning them higher weights, regarding less of the length of the input. Indeed the attention mechanism is invented before Transformer architecture and used in Seq2Seq models achieved with RNN and/or LSTM, which but do not perform well when dealing with long sequence.

b) Q, K and V are namely the Query, Key and Value matrices in self-attention, the idea of them comes from the information retrieval system, where the Query matrix represents the need of a certain pattern in the input embeddings searched by the current processing embedding, while the Key matrix represents the ability of each embedding to satisfy the searched pattern, and the Value matrix represents the effective contribution that each embedding would have on current pattern. Basically all input embeddings are multiplied with each of three matrices, let's call EQ, EK and EV, then EQ and EK are multiplied to compute the importance of each embeddings to others, which will then be softmaxed column-wisely to compute the weights, and their result are multiplied to EV to get the final deviation vector to added to initial embedding vector.

c) In decoder, as during the inference phase, we do not know the words of output sequence after the current processed word, we cannot compute the matchability and attention score of them, therefor in training phase, to perserve the parallel execution for sake of efficiency, instead of training pairs of input output sequence sequentially, we simply mask the future words of current word, by adding a -inf to their multiplication of EQ and EK, so they will not influence current word having a zero weight.

d) The self attention mechanism is permutation invariant, which means when changing only the order of words in input sequence, the output sequence will remain exactly same, except for the orders of the words. In order to make the architecture sensible and variant to the position of the words in input sequence, the positional encoding is introduced, which leverages the sin and cos functions, encodes the length of the sequence, the index of words in the sequence to each of them under the form of a vector, that are summed to the words' embedding.

Comment:

1+1+1+1

Question 5

Partially correct

Mark 0.67 out of 1.00

As simple as it might seem, counting is not a trivial task.

Which of the following networks could be able to learn how to count the number of times the sequence of characters "asdf" is present in a sequence of characters of arbitrary length starting with the <start> token and ending with the <end> token.

The number of characters in the input sequence is not known in advance.

Select one or more:

- ☒ a. A 1D Convolutional network followed by an LSTM with a linear output (to be rounded)  Yes, it can deal with arbitrary length sequences and the output is a number, we assume we can solve the effect of the vanishing gradient with LSTM.
- ☒ b. A Bidirectional LSTM taking the sequence of characters as input and providing as output a number (to be rounded)  Yes, the Bidirectional LSTM input is not limited in size and we assume we can solve the effect of the vanishing gradient with LSTM.
- ☐ c. A 1D Convolutional network with a GAP layer followed by a dense layer with linear activation function (output to be rounded)

Your answer is partially correct.

You have correctly selected 2.

The correct answers are:

A Bidirectional LSTM taking the sequence of characters as input and providing as output a number (to be rounded),

A 1D Convolutional network with a GAP layer followed by a dense layer with linear activation function (output to be rounded),

A 1D Convolutional network followed by an LSTM with a linear output (to be rounded)

Question 6

Partially correct

Mark 1.50 out of 2.00

Let us consider the object detection problem. Mark all the sentences that are correct.

Please, refrain from marking answers that are "technically possible in Python", and stick to those answers that a Neural Network expert (as you are expected to be) would give

Select one or more:

- ☒ a. There are object detection networks that leverage a hand-crafted region proposal algorithm to identify regions that potentially contain objects, then perform inference on these using a classification backbone. ✓ TRUE: R-CNN is done in this way.
- ☒ b. It is in principle possible to implement an object detection network mimicking the architecture of Faster R-CNN to classify heartbeats in ECG tracing, which is 1D signal. ✓ TRUE: the backbone should be conveniently made of 1D filters and the sizes of all the following need to be adjusted.
- ☒ c. Object detection networks can return a varying number of outputs for a single input image. ✓ TRUE, depending on the content of the image
- ☒ d. When performing object or event detection in a time series, it is possible to adopt a MLP taking as input the time series. ✗ FALSE: Fully Connected layers are global and do not consider the spatial dimension of the input
- ☐ e. In Faster R-CNN the anchors identify a few regions in the image that are then fed to a GAP to extract a vector that goes to the output
- ☐ f. Faster R-CNN was the first object detection network leveraging a backbone pre-trained on ImageNet
- ☐ g. If you want to train an R-CNN to detect L different types of objects, you need to train a classifier using L classes, as the objectiveness score will take care of excluding detections on the background.
- ☐ h. The output of Faster R-CNN is actually the output of the Region Proposal Network (RPN), which contains bounding boxes and their class estimates.
- ☐ i. There are many object detection networks, but only those attempting to segment objects have skip connections.

Your answer is partially correct.

You have selected too many options.

The correct answers are:

There are object detection networks that leverage a hand-crafted region proposal algorithm to identify regions that potentially contain objects, then perform inference on these using a classification backbone.,

It is in principle possible to implement an object detection network mimicking the architecture of Faster R-CNN to classify heartbeats in ECG tracing, which is 1D signal.,

Object detection networks can return a varying number of outputs for a single input image.

Question 7

Partially correct

Mark 0.80 out of 1.00

The following questions concern Autoencoders. Select all those sentences that are true.

As in the previous section, act as a neural network expert, do not consider true what is simply possible to program.

Select one or more:

- ☐ a. Like most neural networks, autoencoders require annotated input for training.
- ☐ b. Convolutional autoencoders architectures have been primarily used for generating images of human faces
- ☒ c. Convolutional autoencoders architectures can be trained to restore images where there are missing values ✓ TRUE
- ☐ d. Autoencoders are special types of neural network that take as input an image and returns a vector.
- ☐ e. Autoencoders always need to have a bottleneck for the latent representation.
- ☒ f. Autoencoders can be used to pre-train the backbone for a classifier in case there are little annotated samples available, but many label-free images ✓ TRUE: as we have explained during lectures
- ☐ g. Convolutional autoencoders are made of convolutional, activations and pooling layers. It is not possible to place a Dense layers within convolutional autoencoders.
- ☐ h. Autoencoders are ultra-fast model that can be used in high throughput streams of images to detect known shapes.
- ☒ i. Convolutional autoencoders architectures can be trained to restore images corrupted by noise. ✓ TRUE: we can just synthetically degrade images (e.g. by adding noise) to get training
- ☒ j. Convolutional autoencoders architectures can be trained to restore out of focus images ✓ TRUE

Your answer is partially correct.

You have correctly selected 4.

The correct answers are:

Convolutional autoencoders architectures can be trained to restore images corrupted by noise.,

Autoencoders can be used to pre-train the backbone for a classifier in case there are little annotated samples available, but many label-free images,

Autoencoders always need to have a bottleneck for the latent representation.,

Convolutional autoencoders architectures can be trained to restore out of focus images ,

Convolutional autoencoders architectures can be trained to restore images where there are missing values

Question 8

Correct

Mark 1.00 out of 1.00

Consider the following sequence of layers

```
input_layer = tfkl.Input(shape=(64,64,1),name='input')
conv_1 = tfkl.Conv2D(filters=16, kernel_size=5, strides=1, padding='same', activation='relu', name='conv_1')(input_layer)
conv_2 = tfkl.Conv2D(128,3,1, padding='same', activation='relu', name='conv_2')(conv_1)
conv_3 = tfkl.Conv2D(32,1,1, padding='same', activation='relu', name='conv_3')(conv_2)
mp = tfkl.MaxPooling2D((2,2), name='mp')(conv_3)
conv_4 = tfkl.Conv2D(64,3,1, padding='same', activation='relu', name='conv_4')(mp)
mp = tfkl.MaxPooling2D((2,2), name='mp')(conv_4)
```

What is the receptive field at the central pixel of the activations after **the last mp**?

Select one or more:

- ☐ a. 15x15
- ☐ b. 5x5
- ☐ c. 10x10
- ☐ d. 8x8
- ☐ e. 22x22
- ☒ f. 14 x 14 ✓
- ☐ g. The entire image
- ☐ h. Since there are two maxpooling, it is one quarter of the input size

Your answer is correct.

The correct answer is:

14 x 14

Question 9

Not answered

Not graded

Consider the following code

```
import tensorflow as tf
tfkl = tf.keras.layers
tfk = tf.keras

def get_model_2(input_shape, features):

    # Output Layer
    input_layer = tfkl.Input(shape=input_shape, name='input_layer')

    # Block 1
    c1 = tfkl.Conv2D(64, kernel_size=5, padding='same', activation='relu', name='block_1_conv_1')(input_layer)
    c1 = tfkl.MaxPooling2D(name='block_1_mp')(c1)
    c1 = tfkl.Dropout(0.2, name='block_1_dropout')(c1)
    s1 = tfkl.Conv2D(128, kernel_size=5, padding='same', activation='relu', name='block_1_conv_2')(c1)
    s1 = tfkl.Conv2D(64, kernel_size=5, padding='same', activation='relu', name='block_1_conv_3')(s1)
    s1 = tfkl.Add(name='block_1_add')([c1,s1])

    # Block 2
    c2 = tfkl.Conv2D(128, kernel_size=5, padding='same', activation='relu', name='block_2_conv_1')(s1)
    c2 = tfkl.MaxPooling2D(name='block_2_mp')(c2)
    c2 = tfkl.Dropout(0.2, name='block_2_dropout')(c2)
    s2 = tfkl.Conv2D(256, kernel_size=5, padding='same', activation='relu', name='block_2_conv_2')(c2)
    s2 = tfkl.Conv2D(128, kernel_size=5, padding='same', activation='relu', name='block_2_conv_3')(s2)
    s2 = tfkl.Add(name='block_2_add')([c2,s2])
    s2 = tfkl.GlobalAveragePooling2D(name='block_2_gap')(s2)

    # Output Layer
    output_layer = tfkl.Dense(features, activation="linear", name='output_layer')(s2)

    model = tf.keras.Model(inputs=input_layer, outputs=output_layer)
    model.compile(
        loss = tfk.losses.MeanSquaredError(),
        optimizer = tfk.optimizers.Adam(),
        metrics = [
            tfk.metrics.MeanAbsoluteError(),
            tfk.metrics.MeanAbsolutePercentageError()
        ]
    )
    return model

input_shape = (32,32,3)
features = 6
model = get_model_2(input_shape=input_shape, features=features)
model.summary()
tfk.utils.plot_model(model)
```

The last command returns the model summary. Below you can find a reference for you to work on the model
Please fill in the scheme and upload it. This is a backup option, the following question will be assessed

Layer (type)	Output Shape	Param #
=====		
input_layer (InputLayer)		
block_1_conv_1 (Conv2D)		
block_1_mp (MaxPooling2D)		
block_1_dropout (Dropout)		
block_1_conv_2 (Conv2D)		

block_1_conv_3 (Conv2D)
block_1_add (Add)
block_2_conv_1 (Conv2D)
block_2_mp (MaxPooling2D)
block_2_dropout (Dropout)
block_2_conv_2 (Conv2D)
block_2_conv_3 (Conv2D)
block_2_add (Add)
block_2_gap (GAP2D)
output_layer (Dense)

Total params:
Trainable params:
Non-trainable params:

Model: "model"

Layer (type)	Output Shape	Param #	Connected to
=====			
input_layer (InputLayer)	[(None, 32, 32, 3)]	0	[]
block_1_conv_1 (Conv2D)	(None, 32, 32, 64)	4864	['input_layer[0][0]']
block_1_mp (MaxPooling2D)	(None, 16, 16, 64)	0	['block_1_conv_1[0][0]']
block_1_dropout (Dropout)	(None, 16, 16, 64)	0	['block_1_mp[0][0]']
block_1_conv_2 (Conv2D)	(None, 16, 16, 128)	204928	['block_1_dropout[0][0]']
block_1_conv_3 (Conv2D)	(None, 16, 16, 64)	204864	['block_1_conv_2[0][0]']
block_1_add (Add)	(None, 16, 16, 64)	0	['block_1_dropout[0][0]', 'block_1_conv_3[0][0]']
block_2_conv_1 (Conv2D)	(None, 16, 16, 128)	204928	['block_1_add[0][0]']
block_2_mp (MaxPooling2D)	(None, 8, 8, 128)	0	['block_2_conv_1[0][0]']
block_2_dropout (Dropout)	(None, 8, 8, 128)	0	['block_2_mp[0][0]']
block_2_conv_2 (Conv2D)	(None, 8, 8, 256)	819456	['block_2_dropout[0][0]']
block_2_conv_3 (Conv2D)	(None, 8, 8, 128)	819328	['block_2_conv_2[0][0]']
block_2_add (Add)	(None, 8, 8, 128)	0	['block_2_dropout[0][0]', 'block_2_conv_3[0][0]']
block_2_gap (GlobalAveragePool)	(None, 128)	0	['block_2_add[0][0]']
output_layer (Dense)	(None, 6)	774	['block_2_gap[0][0]']
=====			

Total params: 2,259,142
Trainable params: 2,259,142
Non-trainable params: 0

Question 10

Correct

Mark 4.00 out of 4.00

Remember that this question will be automatically rated, therefore you have to consider the following details very carefully:

- when the activations become a vector, these are stacked along the third dimension (the channel, C). Therefore, if your latent representation becomes a 3 element vector, enter as sizes H:1 W:1 C:3.
- enter digits **only** and do not use "." or "," or "-" or "[]". Say write "1234" and not "1,234" or "1.234". Write "0", not "-" or "[]".
- do not enter formulas, just resulting numbers
- connections, if any have been omitted in this summary

All the numbers have to be entered here, the text in the previous question is just meant for you to design the solution.

Layer (type)	Output Shape			Param #		
=====						
input_layer (InputLayer)	H:	32 ✓	, W:	32 ✓	, C:	3 ✓ Params #: 0 ✓
block_1_conv_1 (Conv2D)	H:	32 ✓	, W:	32 ✓	, C:	64 ✓ Params #: 4864 ✓
block_1_mp (MaxPooling2D)	H:	16 ✓	, W:	16 ✓	, C:	64 ✓ Params #: 0 ✓
block_1_dropout (Dropout)	H:	16 ✓	, W:	16 ✓	, C:	64 ✓ Params #: 0 ✓
block_1_conv_2 (Conv2D)	H:	16 ✓	, W:	16 ✓	, C:	128 ✓ Params #: 204928 ✓
block_1_conv_3 (Conv2D)	H:	16 ✓	, W:	16 ✓	, C:	64 ✓ Params #: 204864 ✓
block_1_add (Add)	H:	16 ✓	, W:	16 ✓	, C:	64 ✓ Params #: 0 ✓
block_2_conv_1 (Conv2D)	H:	16 ✓	, W:	16 ✓	, C:	128 ✓ Params #: 204928 ✓
block_2_mp (MaxPooling2D)	H:	8 ✓	, W:	8 ✓	, C:	128 ✓ Params #: 0 ✓
block_2_dropout (Dropout)	H:	8 ✓	, W:	8 ✓	, C:	128 ✓ Params #: 0 ✓
block_2_conv_2 (Conv2D)	H:	8 ✓	, W:	8 ✓	, C:	256 ✓ Params #: 819456 ✓
block_2_conv_3 (Conv2D)	H:	8 ✓	, W:	8 ✓	, C:	128 ✓ Params #: 819328 ✓
block_2_add (Add)	H:	8 ✓	, W:	8 ✓	, C:	128 ✓ Params #: 0 ✓
block_2_gap (GAP2D)	H:	1 ✓	, W:	1 ✓	, C:	128 ✓ Params #: 0 ✓
output_layer (Dense)	H:	1 ✓	, W:	1 ✓	, C:	6 ✓ Params #: 774 ✓
=====						
Total params:	2259142 ✓					
Trainable params:	2259142 ✓					
Non-trainable params:	0 ✓					

Layer (type) Output Shape Param # Connected to

```

=====
input_layer (InputLayer)  [(None, 32, 32, 3)] 0 []
block_1_conv_1 (Conv2D)   (None, 32, 32, 64) 4864 ['input_layer[0][0]']
block_1_mp (MaxPooling2D) (None, 16, 16, 64) 0 ['block_1_conv_1[0][0]']
block_1_dropout (Dropout) (None, 16, 16, 64) 0 ['block_1_mp[0][0]']
block_1_conv_2 (Conv2D)   (None, 16, 16, 128) 204928 ['block_1_dropout[0][0]']
block_1_conv_3 (Conv2D)   (None, 16, 16, 64) 204864 ['block_1_conv_2[0][0]']
block_1_add (Add)         (None, 16, 16, 64) 0 ['block_1_dropout[0][0]',
                        'block_1_conv_3[0][0]']
block_2_conv_1 (Conv2D)   (None, 16, 16, 128) 204928 ['block_1_add[0][0]']
block_2_mp (MaxPooling2D) (None, 8, 8, 128) 0 ['block_2_conv_1[0][0]']
block_2_dropout (Dropout) (None, 8, 8, 128) 0 ['block_2_mp[0][0]']
block_2_conv_2 (Conv2D)   (None, 8, 8, 256) 819456 ['block_2_dropout[0][0]']
block_2_conv_3 (Conv2D)   (None, 8, 8, 128) 819328 ['block_2_conv_2[0][0]']
block_2_add (Add)         (None, 8, 8, 128) 0 ['block_2_dropout[0][0]',
                        'block_2_conv_3[0][0]']
block_2_gap (GlobalAveragePool (None, 128) 0 ['block_2_add[0][0]']
output_layer (Dense)      (None, 6) 774 ['block_2_gap[0][0]']
=====

```

Total params: 2,259,142

Trainable params: 2,259,142

Non-trainable params: 0

Grading Criteria

Each answers is assessed but not all correct corresponds to points (some are too simple, some are repeated).

Some wrong answers contributes negatively to your final grade, in particular those with very apparent errors (e.g. setting parameters to a gap or a relu)

Therefore, the final grade can be in principle negative.

Question **11**

Correct

Mark 2.00 out of 2.00

Mark all the tasks that the above network can be trained for.

Please, refrain from marking answers that are "technically and eventually possible in Python", and stick to those tasks that a Neural Network expert (as you are expected to be) would use this network for.

Needless to say, assume you are always provided with the required training data for these tasks, while you cannot change the architecture and training options for the above model.

Select one or more:

- ☐ a. Classify the digits from 0 to 5 from a subset of MNIST dataset.

- ☒ b. Given a picture of a car, estimate its price, its maximum speed, and localize it through a bounding box. ✓ This is a six-output regression problem.
- ☐ c. Given a picture of a car from the front, localize centers of each headlight ("faro anteriore") and of indicator lights ("freccie")
- ☐ d. Detecting laptops and tablets, and for each of these return a bounding box
- ☒ e. Given a picture of an office, count how many employees (divided in male/female) how many managers (divided in male/female) and how many tablets and laptops ✓ True, that's a regression problem
- ☐ f. Given a front picture of a car, draw a bounding box around that and predict the brand of the car
- ☒ g. Given the picture of a human face, estimate the position of the tip of ears and the tip of nose. ✓ TRUE: This is a regression problem over 6 image coordinates

Your answer is correct.

The correct answers are:

Given a picture of an office, count how many employees (divided in male/female) how many managers (divided in male/female) and how many tablets and laptops,

Given a picture of a car, estimate its price, its maximum speed, and localize it through a bounding box.,

Given the picture of a human face, estimate the position of the tip of ears and the tip of nose.